Institute for Complex Systems

JOHANNES KEPLER UNIVERSITÄT LINZ

Univ.-Prof. Dr. Daniel Große
DI Simon Reitinger & Katharina Ruep, M.Sc.

7. Mai 2026

Übung 5
zur Vorlesung Rechnerarchitektur

Abgabe bis **Freitag 15 Mai, 2026 10:15** via EPICS: `https://ep.ics.jku.at`.

Aufgabe 1 – Rechnen mit Gleitkommazahlen (3 + 2 + 2 + 2 Punkte)

Betrachte die Dezimalzahlen `A=360,56251` und `B=-10,46875` sowie die im IEEE-754 Single Precision Format gegebenen Zahlen `C=0xB27C0000` und `D=0x4D302000`. *Hinweis: Die Mantisse des Single Percision Format ist 23-bit lang, also auf 23 Stellen runden!*

a) Stelle `A` und `B` im IEEE-754 Single-Precision Format dar (HEX-Darstellung der Bits). Wie groß ist der entstandene Fehler?

b) Wandle `C` und `D` in eine Dezimalzahl um.

c) Berechne das Produkt `C*D` im IEEE-754 Format. Verifiziere dein Ergebnis, indem du es in eine Dezimalzahl umwandelst.

d) Berechne die Summe `C+D` im IEEE-754 Format. Verifiziere dein Ergebnis indem, du es in eine Dezimalzahl umwandelst.

Gib bei allen Aufgaben Zwischenschritte an, sodass der Lösungsweg ersichtlich ist. Verwende dabei sowohl die binäre als auch die hexadezimale Notation.

Aufgabe 2 – Extrema und Zahlenverteilung (2 + 2 + 3 Punkte)

Betrachte das IEEE-754 Format für Single Precision Zahlen. Gib deine Ergebnisse sowohl im IEEE-754-Format (binär) als auch im Dezimalsystem an.

a) Ermittle die kleinste darstellbare positive Zahl `minreal`.

b) Ermittle die größte darstellbare positive Zahl `maxreal`.

c) Ermittle die kleinsten Zahlen ε_1 und ε_2, sodass $1 + \varepsilon_1 > 1$ und $1024 + \varepsilon_2 > 1024$. Was bedeutet der Unterschied zwischen ε_1 und ε_2 für die Berechnungen mit großen Zahlen verglichen mit Berechnungen nahe bei 0?

Aufgabe 3 – Gleitkommazahlen in RISC-V (3 + 3 + 2 Punkte)

Die folgende Funktion führt Berechnungen mit Gleitkommazahlen in RISC-V Assembler aus.

a) Analysiere den Assembler Code. Kommentiere hierfür die einzelnen Assembler-Zeilen mit deren Funktion.

b) Fertige einen C-Code des Algorithmus basierend auf deiner Analyse an.

c) Was ist das Ergebnis im Register `ft0` (bzw. gespeichert an der `result` Adresse)?

```
.data
result: .float  0.0

.text
.globl main

main:
  addi t0, zero, 8
  addi t1, zero, 1
  fcvt.s.wu ft0, t1
  fcvt.s.wu ft2, t1

loop:
  beq t0, t1, end
  fcvt.s.wu ft1, t1
  fdiv.s ft2, ft2, ft1
  fadd.s ft0, ft0, ft2
  addi t1, t1, 1
  jal zero, loop

end:
  la t2, result
  fsw ft0, 0(t2)
```